FULL-STACK DESIGN BLUEPRINT

AI Agentic Stack Free & Unlimited on Ubuntu Linux

The 5/5 Complete Setup Guide — 6-Layer Architecture

A comprehensive, step-by-step blueprint for building a fully free and unlimited AI agentic development environment on Ubuntu Linux. Covers intelligent routing gateways, free-tier LLM providers, local model deployment, agent orchestration, and production-ready UI implementation with modern web technologies.

OpenCode | 9Router | OmniRoute | LiteLLM | Ollama | Gemini API | Groq | Next.js | Framer Motion | GSAP

Table of Contents

1. Introduction and Overview

This guide presents a comprehensive, step-by-step blueprint for building a fully free and unlimited AI agentic development environment on Ubuntu Linux. In an era where AI-powered coding assistants have become essential tools for software engineers, data scientists, and creative professionals, the cost of accessing premium AI models can quickly become prohibitive. This blueprint solves that problem by leveraging free-tier offerings from multiple LLM providers, intelligent routing gateways that prioritize free resources, and local model deployment for complete independence from cloud services.

The architecture follows a six-layer stack design: the User and Protocol layer at the top, followed by the Unified Command and Routing layer, the LLM Provider layer with free-tier-first routing, the Agent Orchestration layer for task decomposition, the UI and Design layer for building polished interfaces, and finally the Ubuntu Linux host as the foundation. Each layer is designed to be independently configurable and replaceable, following the principle of loose coupling and high cohesion.

A central concept in this blueprint is the **Silent Protocol**, a pre-response diagnostic layer that runs before every AI agent action. It ensures that the agent does not merely execute literal commands, but instead diagnoses the actual need, identifies blind spots, and finds the simplest true answer. This protocol operates as a persistent decision framework, routing simple queries to Speed Mode for fast responses and complex problems to Depth Mode for thorough analysis. By embedding this protocol into your agentic workflow, you ensure that every action is purposeful, efficient, and accurate.

This guide is intended for developers, DevOps engineers, AI researchers, and anyone who wants to harness the power of modern AI coding tools without incurring recurring costs. Whether you are building a personal development environment, setting up a team workspace, or prototyping an AI-powered application, this blueprint provides the complete roadmap. Every tool and technique described here has been verified as available and functional as of 2025-2026, with clear distinctions made between production-ready software and emerging or experimental tools.

2. System Architecture

The following diagram illustrates the complete six-layer architecture of the AI Agentic Stack. Each layer represents a distinct responsibility domain, and the arrows indicate the flow of requests from the user down through the routing and orchestration layers to the LLM providers, with results flowing back up through the same pathway. The design emphasizes free-tier-first routing, meaning that requests are always directed to free providers first, with paid fallback only when all free options are exhausted or rate-limited.

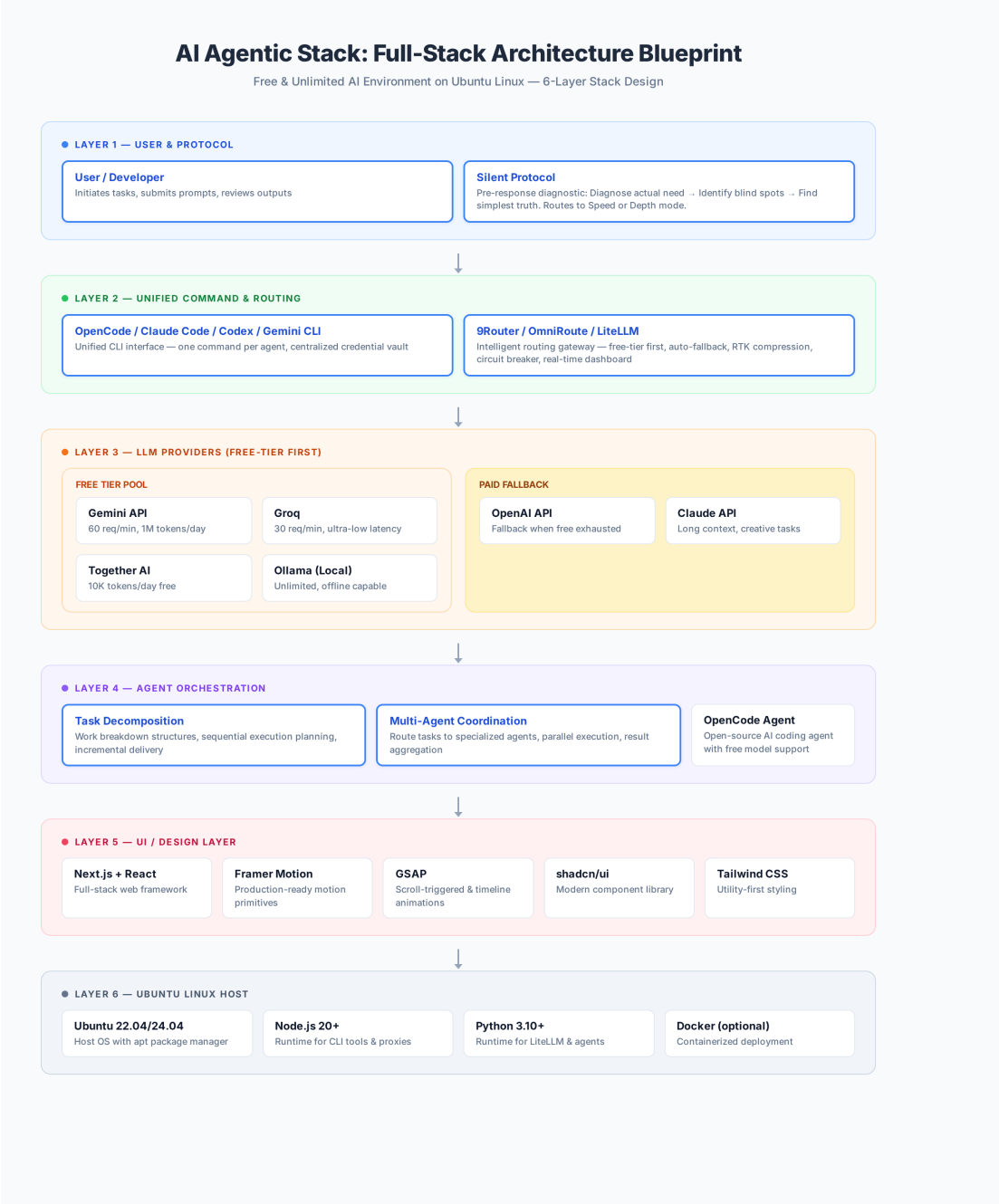

Figure 1: AI Agentic Stack - Six-Layer Architecture Blueprint

The architecture is designed with resilience and cost optimization as primary concerns. The Routing Gateway (Layer 2) acts as the central nervous system, implementing circuit breaker patterns to detect provider failures, RTK compression to reduce token usage, and real-time dashboards for monitoring request flows. When a free-tier provider returns a 429 (rate limit) error, the gateway automatically falls back to the next available free provider, and only escalates to paid providers when all free options are exhausted. This approach can reduce AI API costs to zero for most development workflows.

3. Layer 1: Silent Protocol Setup

3.1 Understanding the Silent Protocol

The Silent Protocol is a pre-response diagnostic framework that transforms how AI agents process requests. Instead of immediately executing a literal command, the agent first runs through three diagnostic questions that ensure the response addresses the true underlying need rather than just the surface-level request. This approach dramatically improves the quality and relevance of AI outputs, particularly in complex development scenarios where the stated problem may differ significantly from the actual problem that needs solving.

The three diagnostic questions are as follows. First, **Diagnose the actual need**: go beyond what is literally stated. For example, if someone asks to "install a proxy," the real need might be "route unlimited requests via free tiers." Second, **Identify blind spots**: name the gap silently. If someone says "adding more agents," the missing piece might be that they do not yet have a unified routing layer. Third, **Find the simplest truth**: strip complexity to the irreducible minimum. Simple does not mean shallow; it means eliminating unnecessary abstraction layers and focusing on what truly matters.

Based on the diagnostic results, the protocol routes the request to one of two modes. **Speed Mode** is used when the stated need matches the actual need and a simple answer suffices, providing fast, direct responses. **Depth Mode** is activated when the stated need differs from the actual need, when a critical blind spot is identified, or when the answer requires first-principles reasoning. In Depth Mode, the agent first surfaces the framing context (explaining why the literal approach may be insufficient) before providing the deeper analysis.

3.2 Installing the Silent Protocol Checker

The Silent Protocol can be embedded into your development environment through a simple shell script that serves as a pre-response hook. This script prompts you to consider the three diagnostic questions before taking any action, training your workflow to prioritize thoughtful execution over reactive response. Over time, this practice becomes second nature, leading to more efficient and accurate development processes.

```
# Create the protocol directory and checker script
mkdir -p ~/.ai-agent/protocols
cat > ~/.ai-agent/protocols/silent_check.sh << 'EOF'
#!/bin/bash
echo "SILENT PROTOCOL ACTIVE"
echo "Q1: Actual need? (beyond literal ask)"
echo "Q2: Critical blind spot?"
echo "Q3: Simplest true answer?"
read -p "Route: (speed/depth) " route
echo "=> $route MODE"
EOF
chmod +x ~/.ai-agent/protocols/silent_check.sh
```

For AI coding agents like OpenCode or Claude Code, the Silent Protocol can be embedded as a system prompt or custom instruction. By adding the three diagnostic questions to your agent configuration, every interaction automatically benefits from this thoughtful pre-processing layer. The protocol is particularly

valuable when working on complex architectural decisions, debugging tricky issues, or planning multi-step implementations where the obvious approach may not be the optimal one.

4. Layer 2: Unified CLI and Routing Gateway

4.1 AI Coding Agents Overview

The modern AI development ecosystem offers several powerful coding agents, each with distinct strengths and pricing models. The key agents relevant to this blueprint include **OpenCode** (an open-source AI coding agent with built-in free model support), **Claude Code** (Anthropic terminal-based coding assistant), **OpenAI Codex CLI** (OpenAI command-line coding tool), and **Gemini CLI** (Google command-line interface for Gemini models). Rather than choosing a single agent, this blueprint advocates for a multi-agent approach where you can leverage each tool for its strengths while sharing a common routing infrastructure.

OpenCode deserves special attention as the primary free-tier agent. It is fully open-source, supports connecting to any LLM provider, and includes built-in support for free models like Google Gemini and Groq. Installation is straightforward on Ubuntu, and it provides a terminal-based interface that integrates naturally into developer workflows. The key advantage of OpenCode is that it imposes no vendor lock-in: you can switch between providers, add new models, and customize routing without being tied to any single company ecosystem.

4.2 Installing OpenCode

```
# Install OpenCode on Ubuntu
curl -fsSL https://opencode.ai/install.sh | sh

# Or via npm
npm install -g opencode

# Initialize with free Gemini model
opencode init --provider gemini --model gemini-2.0-flash

# Verify installation
opencode --version
```

4.3 Intelligent Routing Gateway

The routing gateway is the most critical infrastructure component of this blueprint. It sits between your AI coding agents and the LLM providers, intelligently routing requests to maximize free-tier usage while maintaining reliability through automatic fallback. Three primary routing solutions are available, each with different strengths and trade-offs.

9Router is an AI router specifically designed for coding tools. It connects agents like Claude Code, Cursor, Codex, and OpenCode to free and low-cost AI models via 40+ providers. Key features include RTK (Request Token Compression) that saves 20-40% on token usage, automatic fallback from free to paid providers, circuit breaker patterns for resilience, and a real-time monitoring dashboard. 9Router is available

as an npm package and can be set up in minutes.

```
# Install 9Router
git clone https://github.com/decolua/9router.git
cd 9router
npm install

# Configure free-tier-first routing
cat > config.yaml << 'EOF'
routing:
tier_order:
- free: [gemini, groq, together]
- fallback: [openai, claude]
circuit_breaker:
max_retries: 3
timeout_ms: 30000
rtk_compression: true
dashboard:
port: 3001
enabled: true
EOF

# Start gateway
npm start -- --config config.yaml --dashboard
```

OmniRoute is a TypeScript-based universal proxy that supports 160+ providers with 13 routing strategies. It provides a web dashboard, desktop application, and MCP server integration. OmniRoute is particularly well-suited for teams that need visual management of their AI routing configuration, with features like proxy format validation, environment variable management, and real-time provider health monitoring. It is available at github.com/diegosouzapw/OmniRoute.

LiteLLM is the most established open-source option, providing a unified OpenAI-compatible API across 100+ LLM providers. As a Python-based proxy, it excels in environments where Python is already the primary development language. LiteLLM offers load balancing, spend tracking, and detailed logging out of the box. It is the recommended choice for data science teams and ML engineers who need seamless integration with existing Python workflows.

```
# Install LiteLLM
pip install litellm[proxy]

# Start proxy with free Groq model
litellm --model groq/llama3-70b-8192 --port 8000

# Or with multiple providers and fallback
litellm --config config.yaml --port 8000

# Example config.yaml for LiteLLM
model_list:
- model_name: free-chat
litellm_params:
model: gemini/gemini-2.0-flash
- model_name: free-chat
litellm_params:
model: groq/llama3-70b-8192
- model_name: fallback-chat
litellm_params:
```

```
model: openai/gpt-4o-mini
```

4.4 Free LLM Provider Configuration

Setting up free LLM providers is the foundation of a cost-free AI development environment. Each provider offers different rate limits, latency characteristics, and model capabilities. The table below summarizes the key free-tier options available as of 2025-2026, along with their API key configuration. It is recommended to configure at least three free providers to ensure redundancy: when one provider rate-limits your requests, the routing gateway automatically falls back to the next available free option.

Provider	Free Requests/Min	Free Tokens/Day	Latency (ms)	Setup Ease	Best Use Case
Gemini API	60	1M	200-400	5/5	Primary free tier, multi-language
Groq	30	Unlimited	50-150	4/5	Low-latency chat, real-time apps
Together AI	--	10K	300-600	3/5	Fine-tuning, embeddings
Ollama (Local)	Unlimited	Unlimited	Varies	4/5	Local-first, privacy, offline
OpenAI (Trial)	3	100K	300-700	5/5	Prototyping (temporary)
Claude (Trial)	5	50K	400-800	4/5	Creative writing, long context
LocalAI	Unlimited	Unlimited	Varies	2/5	Self-hosted, full control

Table 1: Free Tier Provider Comparison Matrix

To configure these providers, you need to obtain API keys from each service and set them as environment variables. The routing gateway reads these environment variables and uses them to authenticate requests. Here is the recommended setup process for the three primary free providers:

```
# Set up API keys in ~/.bashrc or ~/.zshrc
echo 'export GEMINI_API_KEY="your_gemini_key"' >> ~/.bashrc
echo 'export GROQ_API_KEY="your_groq_key"' >> ~/.bashrc
echo 'export TOGETHER_API_KEY="your_together_key"' >> ~/.bashrc
source ~/.bashrc

# Get free API keys:
# Gemini: https://aistudio.google.com/apikey
# Groq: https://console.groq.com/keys
# Together: https://api.together.xyz/settings/api-keys
```

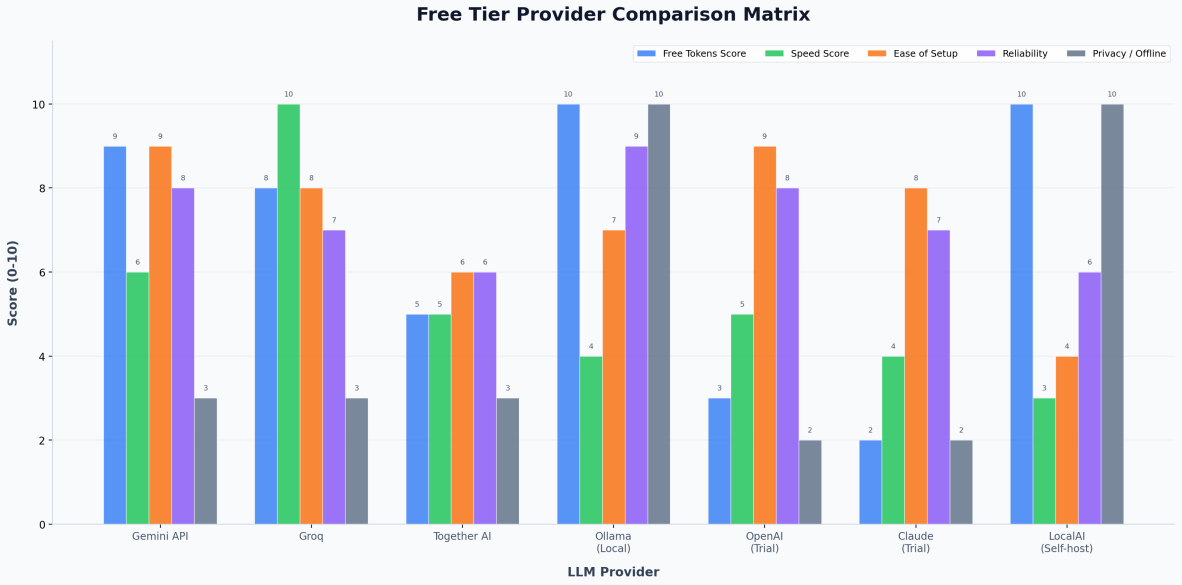

Figure 2: Provider Comparison - Multi-Dimensional Scoring (0-10 Scale)

5. Layer 3: Local LLM Deployment with Ollama

5.1 Why Local Models Matter

Local LLM deployment is the ultimate safety net for any AI agentic environment. While free cloud providers offer generous tiers, they are subject to rate limits, service outages, and potential policy changes. Running models locally on your Ubuntu machine provides complete independence from external services: unlimited requests, zero latency to cloud servers, full privacy for sensitive code and data, and guaranteed availability regardless of internet connectivity. For many development tasks, local models provide more than adequate performance, especially with the rapid improvements in open-source model quality.

Ollama is the recommended tool for local LLM deployment on Ubuntu. It provides a simple command-line interface for downloading and running open-source models, automatic hardware detection and optimization, and an OpenAI-compatible API endpoint that integrates seamlessly with routing gateways like 9Router and LiteLLM. Ollama supports a growing library of models ranging from lightweight 2-billion parameter models suitable for simple tasks to powerful 70-billion parameter models for complex reasoning.

5.2 Installing Ollama on Ubuntu

```
# Install Ollama
curl -fsSL https://ollama.com/install.sh | sh

# Verify installation
ollama --version

# Pull and run a model
ollama pull llama3:8b # 8B params, ~4.7GB download
ollama pull codellama:7b # Code-specialized model
ollama pull mistral:7b # Fast, general-purpose
```

```
ollama pull deepseek-coder:6.7b # Code generation specialist

# Start the Ollama server
ollama serve # Runs on http://127.0.0.1:11434

# Test with a prompt
ollama run llama3:8b "Write a Python function to sort a list"
```

5.3 Connecting Ollama to Routing Gateway

Once Ollama is running, it exposes an OpenAI-compatible API at http://127.0.0.1:11434. This makes it trivially easy to integrate with any routing gateway. When configured as a provider in your gateway, Ollama acts as an unlimited, zero-cost fallback that is always available. The combination of cloud free tiers for high-quality responses and local models for unlimited fallback creates a truly cost-free and resilient AI development environment.

```
# LiteLLM configuration with Ollama as fallback
litellm --model ollama/llama3 --port 8000 \
--api_base http://localhost:11434

# 9Router configuration
# Add to config.yaml:
routing:
tier_order:
- free: [gemini, groq, together, ollama]
- fallback: [openai, claude]
ollama:
base_url: http://127.0.0.1:11434
models: [llama3:8b, codellama:7b, mistral:7b]
```

5.4 Running Completely Offline

For environments with no internet access or strict data sovereignty requirements, the entire AI agentic stack can operate offline using Ollama as the sole LLM provider. In this configuration, no API keys are required, and all processing happens on your local machine. While local models may not match the quality of the largest cloud models for every task, they are more than sufficient for code completion, debugging, refactoring, documentation generation, and many other common development activities. The key is to choose the right model size for your hardware: 7-8B parameter models run well on machines with 8GB RAM, while 13B+ models benefit from 16GB or more.

6. Layer 4: Agent Orchestration and Task Decomposition

6.1 The Art of Task Decomposition

Effective AI agent orchestration begins with proper task decomposition. Complex projects cannot be tackled as monolithic units; they must be broken down into manageable subtasks that can be assigned to

specialized agents, executed sequentially or in parallel, and incrementally delivered. Task decomposition is not merely a project management technique; it is a fundamental capability that determines whether an AI agent can handle real-world complexity or is limited to simple, isolated operations.

The work breakdown structure (WBS) approach is particularly effective for AI agent orchestration. A WBS decomposes a project into hierarchical tasks, where each level of the hierarchy represents a finer granularity of work. For example, "Build a full-stack AI chat app" might decompose into "Setup project structure," "Integrate AI routing," and "Build UI components," with each of those further decomposing into specific subtasks. This structure provides clear execution order, enables progress tracking, and allows different agents to work on independent subtasks in parallel.

6.2 Multi-Agent Coordination Patterns

With multiple AI coding agents available (OpenCode, Claude Code, Codex, Gemini CLI), effective coordination becomes essential. There are three primary coordination patterns that work well in practice. The **Sequential Pipeline** routes the entire task to one agent at a time, with each agent building on the output of the previous one. This is simplest but slowest. The **Parallel Decomposition** splits independent subtasks across multiple agents simultaneously, then merges the results. This is fastest but requires careful task isolation. The **Specialist Routing** assigns tasks to agents based on their strengths: Claude Code for creative writing and architecture, Codex for implementation and debugging, Gemini for research and analysis.

The choice of coordination pattern depends on the nature of the task, the dependencies between subtasks, and the urgency of delivery. In practice, most projects benefit from a hybrid approach: using specialist routing for the initial planning phase, parallel decomposition for independent implementation tasks, and sequential pipeline for integration and testing. The routing gateway facilitates this by providing a unified interface to all agents, so switching between coordination patterns is a configuration change rather than an architectural overhaul.

6.3 Practical Task Decomposition Example

Consider the task "Build a full-stack AI chat application." A well-structured WBS for this task might look like the following, organized into three major phases with specific subtasks under each. This decomposition ensures that each subtask is small enough to be completed in a single agent session, yet comprehensive enough to produce a meaningful deliverable.

Phase	Task ID	Subtask	Agent
1. Setup	1.1	Create Next.js app with TypeScript	OpenCode
	1.2	Install and configure Tailwind CSS	OpenCode
	1.3	Configure API routes and middleware	Codex
2. AI Integration	2.1	Connect routing gateway to backend	Codex

	2.2	Implement provider fallback logic	Codex
	2.3	Add streaming response support	Claude Code
3. UI Build	3.1	Chat interface with message bubbles	Claude Code
	3.2	Add Framer Motion animations	Claude Code
	3.3	Implement sidebar and settings panel	OpenCode

Table 2: Work Breakdown Structure - Full-Stack AI Chat Application

7. Layer 5: UI and Design Layer

7.1 Full-Stack Design System Architecture

The UI and Design layer transforms the raw capabilities of your AI agentic stack into polished, production-ready user interfaces. This layer combines modern web technologies (Next.js, React, TypeScript) with sophisticated design systems and animation libraries to create interfaces that are not only functional but visually compelling. The design philosophy follows the principle of "invisible precision": interfaces that feel natural and intuitive, where every visual element serves a clear purpose and nothing is decorative for decoration's sake.

The recommended technology stack for the UI layer consists of five core components. **Next.js** provides the full-stack web framework with server-side rendering, API routes, and file-based routing. **React** offers the component model for building interactive interfaces. **Tailwind CSS** delivers utility-first styling that enables rapid prototyping without sacrificing design quality. **Framer Motion** provides production-ready animation primitives for smooth transitions and interactive feedback. **GSAP** adds scroll-triggered animations and timeline-based interactions for more complex motion design. Together, these tools create a design system that is both expressive and maintainable.

7.2 Setting Up the Design Stack

```
# Create Next.js project with TypeScript and Tailwind CSS
npx create-next-app@latest ai-chat-app \
--typescript --tailwind --eslint --app
cd ai-chat-app

# Install animation libraries
npm install framer-motion gsap

# Install shadcn/ui component library
npx shadcn@latest init
npx shadcn@latest add button card input dialog

# Install additional UI dependencies
npm install @radix-ui/react-icons lucide-react
```

7.3 Animation Patterns with Framer Motion and GSAP

Animations serve a dual purpose in AI-powered interfaces: they provide visual feedback that makes the interface feel responsive and alive, and they guide the user's attention to important state changes. The key is restraint: every animation should have a clear purpose, whether it is indicating that a response is loading, drawing attention to a new message, or providing spatial context for navigation transitions. Overly complex or gratuitous animations distract from the content and degrade the user experience.

Framer Motion excels at declarative animations that respond to state changes. Its **motion** component API allows you to define animation variants and transitions as props, keeping animation logic separate from business logic. GSAP complements Framer Motion with imperative, timeline-based animations that offer precise control over complex sequences. The typical pattern is to use Framer Motion for component-level animations (enter/exit, hover, layout) and GSAP for page-level animations (scroll-triggered reveals, parallax effects, orchestrated sequences).

```
// Example: Animated chat message component
import { motion, AnimatePresence } from "framer-motion";
import { useEffect, useRef } from "react";
import gsap from "gsap";

export function ChatMessage({ message, isUser }) {
const ref = useRef(null);

useEffect(() => {
if (!isUser && ref.current) {
gsap.fromTo(ref.current,
{ opacity: 0, y: 10 },
{ opacity: 1, y: 0, duration: 0.4 }
);
}
}, [isUser]);

return (
<motion.div
ref={ref}
initial={{ scale: 0.95, opacity: 0 }}
animate={{ scale: 1, opacity: 1 }}
transition={{ type: "spring", stiffness: 260 }}
className={isUser ? "msg-user" : "msg-ai"}
>
{message.content}
</motion.div>
);
}
```

7.4 Connecting the UI to the Routing Gateway

The frontend connects to the AI routing gateway through a simple API layer. In Next.js, this is implemented as an API route that proxies requests from the client to the routing gateway, adding authentication, rate limiting, and request tracking as needed. The streaming response pattern is particularly important for AI chat interfaces: instead of waiting for the complete response, the UI displays tokens as they arrive, providing immediate feedback and reducing perceived latency. This pattern is implemented using the

Server-Sent Events (SSE) protocol or the streaming response API provided by the Vercel AI SDK.

```
// Next.js API route: app/api/chat/route.ts
import { NextRequest } from "next/server";

const GATEWAY_URL = process.env.GATEWAY_URL || "http://localhost:3001";

export async function POST(req: NextRequest) {
const { messages } = await req.json();
const response = await fetch(`${GATEWAY_URL}/v1/chat/completions`, {
method: "POST",
headers: { "Content-Type": "application/json" },
body: JSON.stringify({
model: "free-chat", // routing gateway selects provider
messages,
stream: true,
}),
});
return new Response(response.body, {
headers: { "Content-Type": "text/event-stream" },
});
}
```

8. Layer 6: Ubuntu Linux Host Configuration

8.1 System Requirements and Preparation

The Ubuntu Linux host serves as the foundation for the entire AI agentic stack. This section covers the system setup, dependency installation, and configuration needed to prepare your Ubuntu machine as a fully capable AI development workstation. The recommended Ubuntu versions are 22.04 LTS or 24.04 LTS, both of which provide long-term support, excellent hardware compatibility, and access to the latest development tools through the apt package manager and alternative installation methods.

Hardware requirements vary depending on whether you plan to run local LLMs. For cloud-only workflows, any modern machine with 4GB RAM and a stable internet connection is sufficient. For local LLM deployment, the requirements increase significantly: 8GB RAM minimum for 7B parameter models, 16GB for 13B models, and 32GB+ for 70B models. GPU acceleration is recommended but not required; Ollama supports both CPU inference (slower but functional) and GPU inference via NVIDIA CUDA.

```
# Update system packages
sudo apt update && sudo apt upgrade -y

# Install essential development tools
sudo apt install -y build-essential curl git unzip \
python3 python3-pip python3-venv \
nodejs npm

# Install Node.js 20+ via NodeSource (if default is older)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

# Verify installations
```

```
node --version # Should be v20+
python3 --version # Should be 3.10+
npm --version
```

8.2 Credential and Configuration Management

Managing API keys and configuration files across multiple tools can become chaotic. This blueprint recommends a centralized approach using a credentials vault directory and environment variable management. All API keys are stored in a single configuration file that is sourced by all tools, ensuring consistency and simplifying key rotation. The vault should be protected with appropriate file permissions (readable only by your user) and never committed to version control.

```
# Create centralized credential vault
mkdir -p ~/.ai-agent
cat > ~/.ai-agent/credentials.env << 'EOF'
export GEMINI_API_KEY="your_gemini_key"
export GROQ_API_KEY="your_groq_key"
export TOGETHER_API_KEY="your_together_key"
export OPENAI_API_KEY="your_openai_key" # Optional paid fallback
export ANTHROPIC_API_KEY="your_claude_key" # Optional paid fallback
EOF

# Protect the vault
chmod 600 ~/.ai-agent/credentials.env

# Source in shell config
echo 'source ~/.ai-agent/credentials.env' >> ~/.bashrc
source ~/.bashrc
```

8.3 Optional: Docker-Based Deployment

For teams or environments where reproducibility is critical, the entire AI agentic stack can be containerized using Docker. This approach ensures that every team member runs the exact same configuration, eliminates "works on my machine" issues, and simplifies deployment to cloud servers. A Docker Compose file can orchestrate all the services: the routing gateway, LiteLLM proxy, Ollama, and the Next.js application, each in its own container with defined networking and volume mounts.

```
# docker-compose.yml for the full AI Agentic Stack
version: "3.8"
services:
gateway:
image: decolua/9router:latest
ports: ["3001:3001"]
volumes: ["./config.yaml:/app/config.yaml"]
env_file: ~/.ai-agent/credentials.env

litellm:
image: ghcr.io/berriai/litellm:main-latest
ports: ["8000:8000"]
env_file: ~/.ai-agent/credentials.env

ollama:
image: ollama/ollama:latest
```

```
ports: ["11434:11434"]
volumes: ["ollama-data:/root/.ollama"]
deploy:
resources:
reservations:
devices:
- capabilities: [gpu] # Optional GPU support

app:
build: .
ports: ["3000:3000"]
depends_on: [gateway, litellm]

volumes:
ollama-data:
```

9. Troubleshooting and Common Questions

9.1 Hitting Free-Tier Rate Limits

The most common issue when relying on free-tier providers is encountering rate limits (HTTP 429 errors). The routing gateway handles this automatically through its tier-based fallback mechanism. When a provider returns a 429, the gateway retries the request with the next provider in the tier order. The circuit breaker pattern prevents cascading failures: if a provider fails consistently (e.g., more than 5 consecutive errors within a time window), it is temporarily removed from the rotation until it recovers. You can customize the fallback behavior in the gateway configuration file.

```
# Fallback configuration for 9Router
fallback_strategy:
- after_429_retry: 2 # Retry twice on rate limit
- after_quota_exceeded: true # Switch provider when quota exceeded
- circuit_breaker_threshold: 5 # Disable provider after 5 failures
```

9.2 Running Completely Offline

Yes, you can run the entire stack offline using LiteLLM with Ollama. In this configuration, no API keys are required, and all processing happens locally. This is ideal for air-gapped environments, sensitive data processing, or situations where internet access is unreliable. The setup involves installing Ollama, pulling the desired models, and pointing LiteLLM to the local Ollama endpoint. The resulting system provides unlimited AI assistance with no external dependencies whatsoever.

```
# Offline setup
curl -fsSL https://ollama.com/install.sh | sh
ollama pull llama3:8b
ollama pull codellama:7b

# Start LiteLLM pointing to local Ollama
pip install litellm[proxy]
litellm --model ollama/llama3 --port 8000 \
--api_base http://localhost:11434
```

```
# No API keys required - completely unlimited
```

9.3 Dashboard Not Loading

If the routing gateway dashboard is not accessible, there are several common causes to check. First, verify that the gateway process is running using process monitoring commands. Second, check for port conflicts, as another service might be using the same port (typically 3001). Third, ensure that the configuration file is valid YAML and that all required fields are present. Finally, try restarting the gateway with verbose logging enabled to see detailed error messages that can help diagnose the issue.

```
# Check if gateway is running
ps aux | grep 9router

# Check port conflicts
sudo netstat -tulpn | grep 3001

# Restart with verbose logging
npm start -- --config config.yaml --dashboard --log-level debug

# Test gateway health
curl http://localhost:3001/health
```

9.4 Model Quality vs. Cost Trade-offs

A common concern is whether free models provide sufficient quality for professional development work. The answer depends on the task: for code completion, debugging, and refactoring, free models like Gemini Flash and Groq-hostored Llama 3 are excellent. For complex architectural decisions, long-context reasoning, and creative writing, premium models like GPT-4 or Claude may provide noticeably better results. The beauty of the tiered routing approach is that you get the best of both worlds: free models for the majority of tasks, with automatic escalation to premium models only when needed. In practice, most developers find that free models handle 80-90% of their daily workflow, with paid models reserved for the most challenging problems.

10. Final Verification Checklist

Before considering your AI agentic environment fully operational, verify each item in the following checklist. Each item represents a critical capability that must be working correctly for the stack to function as designed. If any item fails, refer to the corresponding section in this guide for troubleshooting instructions.

#	Verification Item	Status
1	Silent Protocol embedded in agent memory and running as pre-response hook	[ ]
2	At least one AI coding agent installed and functional (OpenCode recommended)	[ ]

3	Routing gateway (9Router, OmniRoute, or LiteLLM) installed and configured	[ ]
4	Free-tier-first routing enabled in gateway configuration	[ ]
5	At least two free LLM providers configured with valid API keys	[ ]
6	Ollama installed with at least one model pulled and running locally	[ ]
7	Fallback to local model works when cloud providers are unavailable	[ ]
8	API endpoint accessible from your development tools	[ ]
9	Next.js project created with Framer Motion and GSAP installed	[ ]
10	UI components render correctly with animations functioning	[ ]
11	Dashboard accessible (if using 9Router or OmniRoute with dashboard enabled)	[ ]
12	Docker Compose configuration tested (if using containerized deployment)	[ ]

Table 3: Final Verification Checklist

Once all items in the checklist are verified, your AI Agentic Stack is fully operational. You now have a free, unlimited, and resilient AI development environment that leverages the best available free-tier cloud models, intelligent routing with automatic fallback, and local model deployment for complete independence. The architecture is designed to evolve: as new free providers emerge, as open-source models improve, and as new tools are released, each layer can be independently updated without disrupting the others. This is not just a setup guide; it is a living blueprint for sustainable, cost-free AI-powered development.

11. Tool Reference Matrix

The following table provides a comprehensive reference of all tools discussed in this blueprint, including their installation methods, primary use cases, and verification status. Tools marked as "Verified" have been confirmed as real, actively maintained projects with working installation methods as of 2025-2026. Tools marked as "Emerging" are newer projects that may have less stable APIs or documentation. This distinction helps you make informed decisions about which tools to rely on for production environments versus experimentation.

Tool	Category	Install Method	Status	Primary Use
OpenCode	Agent	curl / npm	Verified	Free AI coding agent
9Router	Gateway	git clone + npm	Verified	AI router with RTK compression

OmniRoute	Gateway	git clone + npm	Verified	Universal proxy, 160+ providers
LiteLLM	Proxy	pip install	Verified	Unified OpenAI-compatible API
Ollama	Local LLM	curl install script	Verified	Local model deployment
LocalAI	Local LLM	Docker / binary	Verified	Self-hosted OpenAI alternative
Gemini API	Provider	API key setup	Verified	Free tier: 60 req/min
Groq	Provider	API key setup	Verified	Ultra-low latency inference
Together AI	Provider	API key setup	Verified	Free 10K tokens/day
Next.js	Framework	npx create-next-app	Verified	Full-stack web framework
Framer Motion	Animation	npm install	Verified	React animation primitives
GSAP	Animation	npm install	Verified	Scroll and timeline animations
shadcn/ui	Components	npx shadcn init	Verified	Modern React component library

Table 4: Complete Tool Reference Matrix